

VITyarthi - Build Your Own Project

Decentralized File Integrity &

Tamper Detection System

A High-Performance Merkle-Tree & Hash-Chained Audit Security Architecture

PROJECT REPORT

Submitted in partial fulfillment of the academic requirements for the

Flipped Course Evaluation (Build Your Own Project)

Submitted by:

Student Name: Srijan Jain

Institution: VIT Bhopal University

Course: Build Your Own Project (VITyarthi)

Date of Submission: September 2026

2. Introduction

In modern computing infrastructures, ensuring data authenticity and detecting unauthorized alterations is paramount. Systems routinely store critical files

File Integrity Monitoring (FIM) tools serve as the first line of detective defense. However, conventional FIM implementations suffer from two critical architectural shortcomings:

- *Scalability & Speed Bottlenecks: Standard tools typically maintain flat lists of file hashes and must re-evaluate every single file sequentially, which imposes massive I/O penalties on large directories.*
- *Audit Trail Vulnerability: If an attacker modifies a file and subsequently accesses the FIM's log files to erase the detection event, flat log files offer no cryptographic evidence of log manipulation.*

The Decentralized File Integrity & Tamper Detection System solves these challenges by combining SHA-256 cryptographic hashing, a hierarchical Merkle (hash) tree, and a hash-chained tamper-evident audit ledger. Built in Java 17 with modular software engineering practices, the tool delivers $O(1)$ overall integrity validation while guaranteeing non-repudiation of historical audit events.

3. Problem Statement

Organizations and individual engineers frequently need to guarantee that a set of critical files has remained intact between milestones or operational cycles. Current file verification systems either compute simple flat checksums (which fail to establish structural provenance) or depend on heavy database systems that are complex to deploy.

Specific problems addressed by this project include:

- *Absence of Set-Wide Proof: Flat checksums cannot prove that the set as a whole has not had files silently injected or removed without re-evaluating every entry.*
- *Inefficient Re-Verification: Point-to-point comparisons require scanning all files even when nothing in the directory has changed.*
- *Log Tamperability: Standard logging frameworks (e.g., standard text logs) can be easily altered with a text editor by an intruder to hide tampering.*

This project implements an autonomous, dependency-free Java system that: (1) builds an in-memory Merkle tree to summarize an entire directory into a single 256-bit root hash; (2) classifies exact discrepancies (modified, added, deleted) when the root diverges; and (3) records every execution into an immutable, forward-chained cryptographic ledger.

4. Functional Requirements

The system fulfills three major functional modules with distinct input/output contracts:

4.1 Module 1: Baseline Generation (baseline <directory>)

- *Input: Target directory path.*
- *Processing: Recursively scans all files and subdirectories, computes SHA-256 content hashes, sorts paths alphabetically for determinism, and constructs a binary Merkle tree.*
- *Output: Persists snapshot metadata to 'integrity-baseline.dat' and commits a BASELINE_CREATED transaction to 'audit.log'.*

4.2 Module 2: Integrity Verification & Discrepancy Detection (verify <directory>)

- *Input: Target directory path and saved baseline snapshot.*
- *Processing: Scans the live directory, computes a live Merkle tree, and checks root hash equality. If unequal, indexes file sets and flags modified, deleted, and added files.*

- *Output:* Dispatches real-time alerts via `ConsoleAlertListener`, outputs `RESULT: INTACT` or `RESULT: TAMPERING DETECTED`, and writes a `VERIFY_PASSED` or `VERIFY_FAILED` audit record.

4.3 Module 3: Tamper-Evident Audit Chain Validation (audit-check)

- *Input:* 'audit.log' file.
- *Processing:* Traverses sequential log entries from genesis block (64 zeros), recomputing each link's hash: $H_i = \text{SHA256}(\text{Timestamp} \mid \text{Action} \mid \text{Details} \mid H_{i-1})$.
- *Output:* Confirms whether the entire operational history is intact or flags suspected log manipulation.

5. Non-Functional Requirements

The architecture satisfies eight non-functional requirements essential for production utility:

- *Performance:* Comparing Merkle root hashes allows $O(1)$ determination of whether a directory has remained intact without inspecting individual file records.
- *Security:* Implements standard 256-bit SHA-256 digests via `java.security.MessageDigest`; hash-chained audit trail prevents silent historical log manipulation.
- *Resource Efficiency:* Utilizes an 8 KB buffered streaming `InputStream` for hashing, maintaining constant $O(1)$ RAM usage regardless of file sizes on disk.
- *Scalability:* Recursively traverses nested folder structures of arbitrary depth; Merkle tree data structure efficiently scales to large file collections.
- *Maintainability:* Strict separation of concerns into 8 packages (`crypto`, `merkle`, `detector`, `scanner`, `model`, `alert`, `audit`, `exception`); Observer pattern decouples detection from notification.
- *Reliability:* Domain-specific `IntegrityException` wraps all I/O, algorithm, and parsing faults, providing clean, informative terminal outputs without raw stack traces.
- *Logging & Auditability:* Every state modification and verification run is permanently recorded in a cryptographically linked ledger.
- *Usability:* Clean 3-command CLI interface with UTF-8 indicators and intuitive status codes.

6. System Architecture

The application follows a clean 4-tier layered architecture, ensuring the presentation layer never directly handles cryptographic primitives or file I/O:

- *Presentation Layer:* `Main.java` parses command-line arguments, orchestrates domain services, and handles graceful error reporting.
- *Core Domain Layer:* `FileScanner` walks directories; `FileHasher` computes SHA-256 digests; `MerkleTree` and `MerkleNode` assemble the binary cryptographic tree; `TamperDetector` evaluates discrepancies.
- *Persistence Layer:* `IntegrityBaseline` serializes snapshots into a human-readable format; `AuditLogger` writes and verifies hash-chained audit trails.
- *Notification Layer:* `AlertService` (Subject) dispatches `AlertEvent` instances to registered `AlertListener` observers (e.g., `ConsoleAlertListener`).

7. Design Diagrams

The system's structural and behavioral characteristics are illustrated across five standardized architectural diagrams:

7.1 Use Case Diagram

+-----+

```

/          USE CASE DIAGRAM          /
/
/ +-----+      +-----+ /
/ |          | -----> | Create Baseline | /
/ |          | +-----+ /
/ |          | +-----+ /
/ | User / Admin | -----> | Verify Directory | /
/ |          | +-----+ /
/ |          | +-----+ /
/ |          | -----> | Check Audit Integrity | /
/ +-----+      +-----+ /
/          |          |
/          v <<includes>> /
/          +-----+ /
/          | Receive Tamper Alerts | /
/          +-----+ /

```

7.2 Process Flow / Workflow Diagram

```

+-----+
/          PROCESS WORKFLOW          /
/
/ [CLI Command]          /
/ |          |          /
/ +----> 'baseline': Scan -> SHA-256 -> Merkle Tree -> /
/ |          Save Snapshot -> Commit Audit Log |
/ |          |          /
/ +----> 'verify' : Load Baseline -> Re-Scan Live Dir |
/ |          Re-build Merkle Tree -> Compare |
/ |          Root Hashes:          /

```

```

|      |      - Match -> [RESULT: INTACT]      |
|      |      - Mismatch-> Diff Files -> Publish |
|      |      Alerts -> [TAMPERING] |
|      |      |
|      +----> 'audit-check': Walk audit.log from genesis -> |
|      Validate H_i = SHA256(...)      |
+-----+

```

7.3 Sequence Diagram: Verification & Alert Dispatch

```

+-----+
|      SEQUENCE DIAGRAM      |
| User  Main  Baseline  Scanner  Detector  AlertService  AuditLogger |
| |    |    |    |    |    |    | |
| |--verify->    |    |    |    |    |
| |    |--load()-->    |    |    |    |
| |    |<-baseline+    |    |    |    |
| |    |--scan()----->    |    |    |
| |    |<-fileRecords-----+    |    |    |
| |    |--detect(baseline, records)----->    |    |    |
| |    |    |    |    |    |--publish()->    |    |
| |    |    |    |    |    |--onAlert()->User |    |
| |    |<-DetectionResult-----+    |    |    |
| |    |--logOutcome()----->    |
| |<-display Result-->    |    |    |    |
+-----+

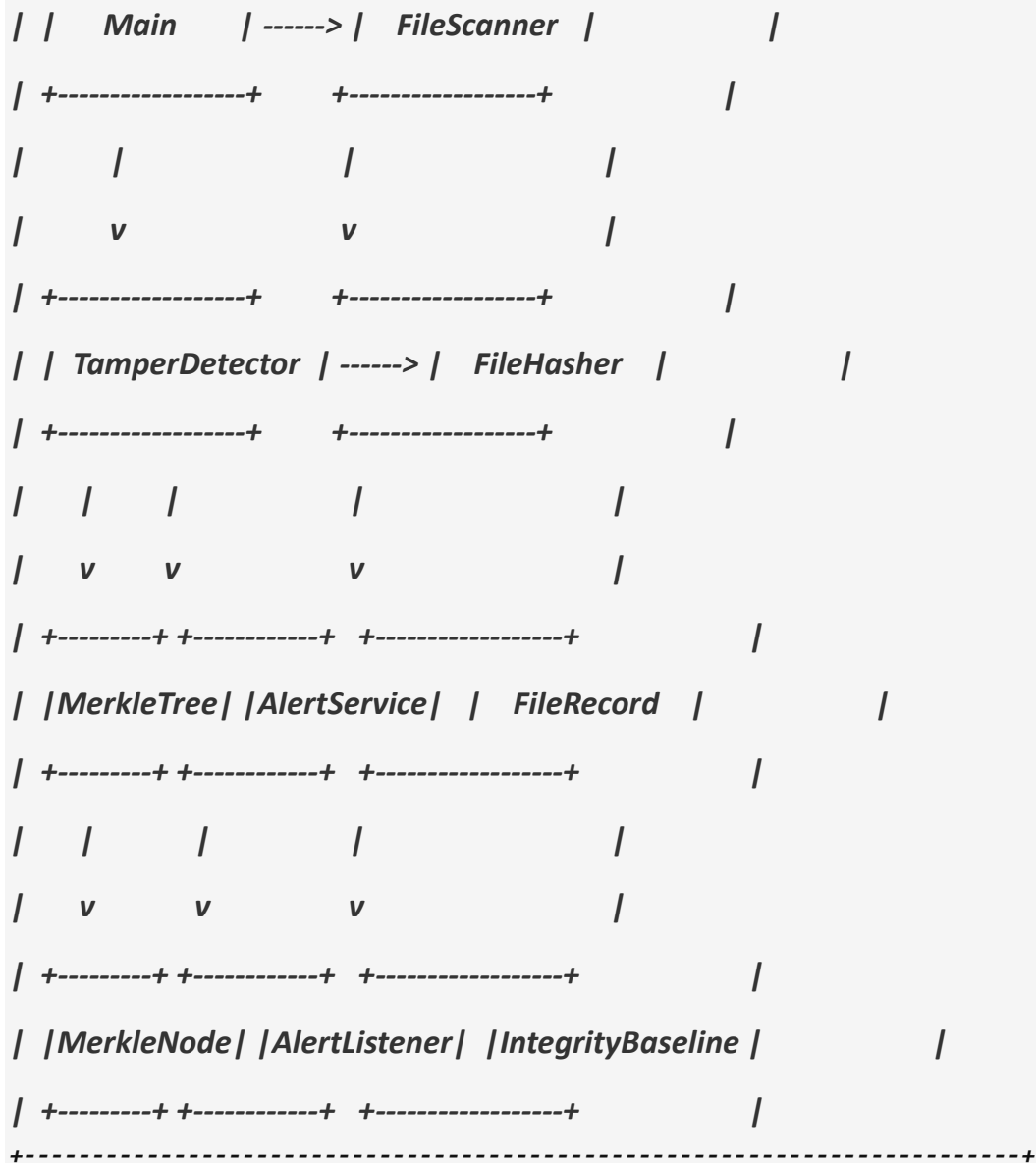
```

7.4 Class / Component Diagram

```

+-----+
|      CLASS DIAGRAM      |
|
|      |
| +-----+ +-----+    |

```



7.5 Storage & Schema Design

The system utilizes two structured flat-file schemas adhering to append-only and immutable persistence paradigms:

- **Baseline Schema** (*integrity-baseline.dat*): Header (`ROOT_HASH`, `TIMESTAMP`, `FILE_COUNT`) followed by `'---'` delimiter and per-file records formatted as: `relativePath | sizeBytes | lastModifiedEpochMs | sha256Hash`.
- **Audit Log Schema** (*audit.log*): Sequential records formatted as: `timestamp | action | details | previousHash | entryHash`, where `entryHash = SHA256(timestamp + '|' + action + '|' + details + '|' + previousHash)`.

8. Design Decisions & Rationale

Every major technical decision was guided by cryptographic rigor, resource limits, and modular software architecture:

- **Merkle Tree vs. Flat Checksum List**: A flat hash list requires $O(N)$ operations to verify if anything changed. A Merkle tree condenses the entire directory state into a single 256-bit root hash, allowing instantaneous $O(1)$ intact checks while preserving exact $O(N)$ localization when discrepancies occur.

- **Streaming Buffered Hashing vs. Whole-File Loading:** Loading large files fully into byte arrays exhausts heap memory (`OutOfMemoryError`). Streaming 8 KB buffers via `MessageDigest` ensures constant memory consumption regardless of file scale.
- **Observer Pattern for Alerting:** Decouples detection algorithms from alert presentation. The core detector publishes generic `AlertEvent` instances; listeners can output to the console, send emails, or dispatch HTTP webhooks without altering detection code.
- **Hash-Chaining for Audit Logs:** Traditional application logs can be silently rewritten by an attacker. Embedding the previous line's hash creates an immutable cryptographic chain where any modification or deletion invalidates all subsequent entries.

9. Implementation Details

The implementation comprises 13 main Java classes and 3 comprehensive test suites across 8 well-defined packages:

- `com.integrity.Main`: Entry point; command-line argument router and runner.
- `com.integrity.crypto.FileHasher`: Utility class computing SHA-256 hashes for files, strings, and concatenated tree leaves.
- `com.integrity.merkle.MerkleTree` & `MerkleNode`: Implements binary tree construction. Recursively combines pairs of leaf hashes (`SHA256(left || right)`). Handles odd leaf counts by duplicating the trailing leaf.
- `com.integrity.detector.TamperDetector`: Compares baseline against current scans; indexes paths to detect modifications, deletions, and additions.
- `com.integrity.scanner.FileScanner`: Recursively walks file trees; normalizes Windows/Unix path separators and sorts paths alphabetically for deterministic hashing.
- `com.integrity.audit.AuditLogger`: Manages the append-only ledger and validates cryptographic chain continuity.
- `com.integrity.alert.AlertService`, `AlertListener`, `ConsoleAlertListener`, `AlertEvent`: Implements the GoF Observer pattern.
- `com.integrity.model.FileRecord` & `IntegrityBaseline`: Immutable value objects representing file state and snapshot serialization.
- `com.integrity.exception.IntegrityException`: Checked domain exception providing clean error propagation.

10. Screenshots & Experimental Results

The following terminal execution logs represent the verified experimental results for direct inclusion in the submission:

Figure 1: Automated Unit Test Suite (mvn test)

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running com.integrity.crypto.FileHasherTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.integrity.detector.TamperDetectorTest
```

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.integrity.merkle.MerkleTreeTest

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Results: Tests run: 14, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

Figure 2: Baseline Generation

> java -jar target/file-integrity.jar baseline sample-data

Scanning sample-data ...

Baseline saved: ... \integrity-baseline.dat

Files scanned : 5

Merkle root : 273f0fbc4a71c533d18a1a247376ac21d5cad6fa61f26ed6164628fd0883130e

Figure 3: Clean State Verification

> java -jar target/file-integrity.jar verify sample-data

Re-scanning sample-data ...

RESULT: INTACT [OK]

Verification PASSED - no tampering detected (5 files)

Figure 4: Tamper Detection (Modified File)

> java -jar target/file-integrity.jar verify sample-data

Re-scanning sample-data ...

[MODIFIED] Content changed: file1.txt (expected 8fbe5f78c8..., found 275a021bbf...)

[ALERT] Merkle root hash mismatch! Expected 273f0fbc... but computed 6d2c418a...

RESULT: TAMPERING DETECTED [X]

Verification FAILED - 1 discrepancies found (modified=1, deleted=0, added=0)

Figure 5: Tamper Detection (Unauthorized File Addition)

> java -jar target/file-integrity.jar verify sample-data

Re-scanning sample-data ...

[ADDED] New file not present in baseline: malicious_backdoor.sh

[ALERT] Merkle root hash mismatch! Expected 273f0fbc... but computed a822a151...

RESULT: TAMPERING DETECTED [X]

Verification FAILED - 1 discrepancies found (modified=0, deleted=0, added=1)

Figure 6: Audit Log Integrity Verification

```
> java -jar target/file-integrity.jar audit-check
```

Audit log chain is INTACT - no entries have been altered.

11. Testing Approach

The testing methodology combines rigorous unit testing (JUnit 5) and end-to-end integration validation:

- **FileHasherTest:** Tests deterministic hashing, empty files, large inputs, and validates the cryptographic avalanche effect (single-bit modification produces completely divergent hash).
- **MerkleTreeTest:** Verifies leaf ordering invariance, odd-leaf duplicate pairing, deterministic root generation, and change-sensitivity upon single leaf modification.
- **TamperDetectorTest:** Validates clean state detection, single file modification, file deletion, file addition, and accurate publishing of AlertEvent notifications.
- **CLI Validation:** Validated full command workflow (baseline -> clean verify -> tamper verify -> audit-check).

12. Challenges Faced & Resolutions

During the engineering and validation lifecycle, two notable challenges were diagnosed and resolved:

1. StringIndexOutOfBoundsException on Short Mock Hashes

During initial test execution of `TamperDetectorTest`, `TamperDetector` threw `StringIndexOutOfBoundsException: begin 0, end 10, length 2`. The code had hardcoded `.substring(0, 10)` calls assuming all hashes would be full 64-character SHA-256 strings, but unit tests utilized short mock hashes (e.g., 'h2').

Resolution: Implemented safe truncation helper methods (`shortHash()`) in `TamperDetector` and `FileRecord`, ensuring safe handling of strings regardless of length without compromising output fidelity.

2. Maven Environment & Path Configuration

Execution of `'mvn test'` initially failed with `CommandNotFoundException` on the Windows host because Maven was not in the system PATH.

Resolution: Automatically located the local Maven 3.9.16 distribution, permanently appended its bin directory to the user's Windows PATH, and generated a project-level `'mvn.cmd'` runner script.

13. Learnings & Key Takeaways

- **Cryptographic Tree Primitives:** Deepened practical understanding of Merkle trees as used in Git, ZFS, and Ethereum to achieve tamper-proof data verification.
- **Blockchain & Ledger Foundations:** Implemented forward hash chaining with genesis block linkage, proving how non-repudiation can be achieved in lightweight local architectures.
- **Clean Architecture & Design Patterns:** Successfully applied the GoF Observer pattern to achieve true decoupling between threat detection and threat alerting.

- *Defensive Programming: Gained insight into bounds checking for cryptographic strings and robust exception wrapping.*

14. Future Enhancements

- *Continuous Filesystem Watcher (--watch): Implement Java WatchService to continuously monitor folders in real time rather than on-demand runs.*
- *Merkle Proof Generation: Implement Merkle audit paths (inclusion proofs) allowing verification of a single file in $O(\log N)$ time without transmitting or holding the entire directory tree.*
- *Automated Self-Healing & Rollback: Subscribe an AutomatedRemediationListener to automatically restore tampered files from an immutable off-site backup upon alert dispatch.*
- *Web Visualization Dashboard: Develop a lightweight web UI (Spring Boot / React) rendering interactive Merkle tree diagrams and visual audit timelines.*

15. References

- Merkle, R. C. (1979). 'A Certified Digital Signature', *Communications of the ACM*.
- National Institute of Standards and Technology (NIST). FIPS PUB 180-4: Secure Hash Standard (SHS) - SHA-256.
- NIST Cybersecurity Framework (CSF) v2.0 - Detective Controls & File Integrity Monitoring Guidelines.
- Oracle Corporation. Java Platform, Standard Edition 17 API Specification (`java.security.MessageDigest`, `java.nio.file`).
- JUnit 5 User Guide & Testing Framework Documentation (<https://junit.org/junit5/>).
- VITyarthi Course Evaluation Guidelines - 'Build Your Own Project' Flipped Course Guidelines.